

Practical-11

Aim

To implement a distributed SMS spam classification system using Apache Spark and Logistic Regression with mini-batch gradient descent, perform text preprocessing using TF-IDF, tune model hyperparameters, evaluate classification performance, and compare training time and accuracy for different dataset sizes and training strategies.

Problem Definition :

You are a Big Data Engineer at SecureComm Inc., a telecom company that processes over 50 million SMS messages daily. The fraud prevention team wants an automated spam classifier that can scale to handle millions of new messages per hour. You will implement a Distributed Logistic Regression model using Mini-batch Gradient Descent on Apache Spark, train it on a massive SMS dataset, and deploy it for batch prediction.

Objectives

1. To load a large-scale SMS spam dataset from HDFS into a Spark DataFrame.
2. To preprocess text data using tokenization, stop-word removal, and TF-IDF feature extraction.
3. To implement Logistic Regression using Spark and mini-batch gradient descent.
4. To tune hyperparameters such as regParam, maxIter, and elasticNetParam.
5. To evaluate the classifier using Accuracy, Precision, Recall, F1-score, and ROC-AUC.
6. To analyze the effect of increasing dataset size on training time and model accuracy.
7. To compare the performance of full-batch and mini-batch gradient descent.

Objective/Outputs:

Tasks: 1. Load the large-scale SMS spam dataset from HDFS into a Spark DataFrame.

```
gitanshu@Parth:~$ mkdir -p ~/sms_spam_practical/data
gitanshu@Parth:~$ cd ~/sms_spam_practical/data
gitanshu@Parth:~/sms_spam_practical/data$ wget -O SMSSpamCollection \
https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip
--2026-10-09 10:04:01-- https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip
Resolving archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252, 64:ff9b::80c3:afc
Connecting to archive.ics.uci.edu (archive.ics.uci.edu)[128.195.10.252]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified
Saving to: 'SMSSpamCollection'

SMSSpamCollection          [  <=>          ] 198.65K  129KB/s   in 1.5s

2026-10-09 10:04:04 (129 KB/s) - 'SMSSpamCollection' saved [203415]
```

```
26/10/09 13:24:12 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, localhost, 40113, None)
Total SMS messages: 5171
+-----+
|label|message|
+-----+
|ham  |Dear, will call Tmorrow.pls accomodate.|
|ham  |Yeah sure, give me a couple minutes to track down my wallet|
|ham  |Good afternoon, my love! How goes that day ? I hope maybe you got some leads on a job. I think of you, boytoy and send you a passionate kiss from acr
oss the sea|
|ham  |Geeeee ... I love you so much I can barely stand it|
|ham  |S'fine. Anytime. All the best with it.|
|ham  |Not getting anywhere with this damn job hunting over here!|
|ham  |Thank you so much. When we skyved wit kz and sura, we didnt get the pleasure of your company. Hope you are good. We've given you ultimatum oh! We are
countin down to aburo. Enjoy! This is the message i sent days ago|
|ham  |Doesn't g have class early tomorrow and thus shouldn't be trying to smoke at &lt;#&gt;|
|ham  |Was doing my test earlier. I appreciate you. Will call you tomorrow.|
|ham  |I have a rather prominent bite mark on my right cheek|
+-----+
only showing top 10 rows
+-----+
|label|count|
+-----+
|ham  |4518|
|spam |653|
+-----+
```

2. Perform text preprocessing: tokenization, stop word removal, and TF-IDF feature extraction.

```
Testing records: 1001
+-----+-----+
|message|features|
+-----+-----+
|&lt;#&gt; in mca. But not conform.|[16384,[334,751,6643,7196],[3.1264249413555243,3.12097533...|
|&lt;#&gt; mins but i had to stop somewhere first.|[16384,[334,751,5660,7385,8611,13331],[3.1264249413555243...|
|and picking them up from various points|[16384,[970,10311,13527],[5.5026977501407295,6.0333260012...|
|came to look at the flat, seems ok, in his 50s? * Is awa...|[16384,[308,467,1575,1589,4774,4851,6346,9129,10771,11990...|
|gonna let me know cos comes bak from holiday that day. ...|[16384,[1391,3445,4167,8273,8301,9499,9859,10887,12409,14...|
+-----+-----+
only showing top 5 rows
TF-IDF feature extraction completed.
(pyspark-env) gitanshu@Parth:~/sms_spam_practical/data$ |
```

3. Implement Logistic Regression using Spark MLlib with mini-batch gradient descent.

```
===== TASK 3 =====
Mini-batch training time: 19.555 seconds
Metrics: {'accuracy': 0.9600399600399601, 'precision': 1.0, 'recall': 0.7202797202797203, 'f1': 0.8373983739837398, 'auc': 0.9670399530539391, 'tp': 103, 't
n': 858, 'fp': 0, 'fn': 40}
```

4. Tune hyperparameters: regularization parameter (regParam), maxIter, and elasticNetParam.

```

===== TASK 4 =====

AUC: 0.9716 regParam: 0.1 maxIter: 40 elasticNetParam: 0.0 seconds: 22.274
AUC: 0.9695 regParam: 0.001 maxIter: 40 elasticNetParam: 0.5 seconds: 22.223
AUC: 0.9694 regParam: 0.01 maxIter: 40 elasticNetParam: 0.0 seconds: 26.329
AUC: 0.9691 regParam: 0.001 maxIter: 40 elasticNetParam: 0.0 seconds: 21.36
AUC: 0.969 regParam: 0.001 maxIter: 40 elasticNetParam: 1.0 seconds: 21.079
AUC: 0.9637 regParam: 0.1 maxIter: 20 elasticNetParam: 0.0 seconds: 12.859
AUC: 0.9628 regParam: 0.01 maxIter: 20 elasticNetParam: 0.0 seconds: 10.549
AUC: 0.9627 regParam: 0.001 maxIter: 20 elasticNetParam: 0.0 seconds: 11.605
AUC: 0.9624 regParam: 0.001 maxIter: 20 elasticNetParam: 0.5 seconds: 10.897
AUC: 0.9623 regParam: 0.001 maxIter: 20 elasticNetParam: 1.0 seconds: 10.858
AUC: 0.9615 regParam: 0.01 maxIter: 40 elasticNetParam: 0.5 seconds: 24.642
AUC: 0.9584 regParam: 0.01 maxIter: 40 elasticNetParam: 1.0 seconds: 23.943
AUC: 0.957 regParam: 0.01 maxIter: 20 elasticNetParam: 0.5 seconds: 10.559
AUC: 0.9499 regParam: 0.01 maxIter: 20 elasticNetParam: 1.0 seconds: 10.505
AUC: 0.8904 regParam: 0.1 maxIter: 20 elasticNetParam: 0.5 seconds: 10.914
AUC: 0.8878 regParam: 0.1 maxIter: 40 elasticNetParam: 0.5 seconds: 22.881
AUC: 0.7255 regParam: 0.1 maxIter: 40 elasticNetParam: 1.0 seconds: 22.665
AUC: 0.6391 regParam: 0.1 maxIter: 20 elasticNetParam: 1.0 seconds: 10.821
Best parameters: 0.1 40 0.0

```

5. Evaluate the model: Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

```

===== TASK 5 =====
Accuracy : 0.956
Precision: 1.0
Recall   : 0.6923
F1-score : 0.8182
ROC-AUC  : 0.9716
TP, TN, FP, FN: 99 858 0 44

```

6. Demonstrate how increasing dataset size affects training time vs accuracy.

```

===== TASK 6 =====
Rows: 200 | Seconds: 14.448 | Accuracy: 0.9061
Rows: 500 | Seconds: 9.764 | Accuracy: 0.9271
Rows: 1000 | Seconds: 15.828 | Accuracy: 0.9421
Rows: 4170 | Seconds: 58.258 | Accuracy: 0.953

```

7. Compare training time: full-batch gradient descent vs mini-batch gradient descent.

```

===== TASK 7 =====
Full-batch time: 85.121 seconds
Full-batch accuracy: 0.953
Mini-batch time: 22.765 seconds
Mini-batch accuracy: 0.956
(pyspark-env) gitanshu@Parth:~/sms_spam_practical/data$
(pyspark-env) gitanshu@Parth:~/sms_spam_practical/data$

```

Key Questions / Analysis

Q1. What is the difference between full-batch, mini-batch, and stochastic gradient descent? Why is mini-batch preferred in distributed settings?

Answer:

Gradient Descent is an optimization algorithm used to minimize the loss function by updating model parameters.

Type	Data used per update	Advantages	Disadvantages
Full-batch GD	Entire training dataset	Stable gradient and updates	High memory and computation cost per update
Mini-batch GD	A small group of training samples	Efficient, scalable, and balances stability with speed	Gradient updates are somewhat noisy
Stochastic GD (SGD)	One training sample	Frequent updates and low memory per update	Highly fluctuating updates

Why is mini-batch preferred in distributed settings?

- Data can be divided among multiple workers.
- Workers can calculate gradients on smaller batches in parallel.
- It reduces the amount of data processed in each update compared with full-batch GD.
- It offers a practical balance between computational efficiency and convergence stability.

Q2. How does the TF-IDF vectorizer handle the curse of dimensionality for text classification?

Answer:

The curse of dimensionality occurs when text data produces a very large number of features because every unique word may become a separate feature.

TF-IDF (Term Frequency–Inverse Document Frequency) assigns weights to words according to their importance in a document.

$$\text{TFIDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

- Term Frequency (TF): Measures how frequently a word appears in a document.
- Inverse Document Frequency (IDF): Reduces the importance of words that occur in many documents.

TF-IDF alone does not eliminate high dimensionality. In Spark, HashingTF can map words into a fixed-size feature vector, reducing vocabulary storage requirements. Sparse vectors store mostly nonzero values efficiently.

However, hashing may cause collisions, and the curse of dimensionality is reduced rather than completely eliminated.

Q3. What is L1 vs L2 regularization? How does elasticNetParam blend them in Spark MLlib?

Answer:

Regularization prevents overfitting by penalizing excessively large model coefficients.

- L1 regularization (Lasso): Penalizes the absolute values of coefficients and can force some coefficients to zero. It performs feature selection.
- L2 regularization (Ridge): Penalizes the squared values of coefficients. It generally shrinks coefficients toward zero without making them exactly zero.

Spark MLlib uses elasticNetParam to control the mixture of L1 and L2 regularization.

elasticNetParam	Regularization
0.0	Pure L2
0.5	Equal mixture of L1 and L2
1.0	Pure L1

The regParam parameter controls the overall strength of the penalty. A higher value generally applies stronger regularization.

Q4. In a distributed environment, how do gradient updates from multiple workers get aggregated? What is the risk of stale gradients?

Answer:

In distributed gradient descent, training data is divided among multiple workers. Each worker calculates a local gradient using its assigned data.

The gradients are then aggregated, commonly by averaging them:

The optimizer uses the aggregated gradient to update the model parameters.

Risk of stale gradients: In asynchronous training, a worker might calculate a gradient using an older version of the model parameters. If that gradient is applied after other workers have updated the model, it may no longer represent the current loss function accurately.

This can slow convergence, cause unstable updates, or reduce training performance. Synchronous updates, parameter versioning, and bounded-staleness techniques can help manage this problem.

Q5. If Recall is more important than Precision for spam detection, how would you adjust the classification threshold? Show with examples.

Answer:

The classification threshold determines when a message is labelled as spam. The default threshold is commonly 0.5.

If Recall is more important than Precision, lower the threshold. This allows more messages to be classified as spam, helping detect more actual spam messages but potentially increasing false positives.

Supplementary Problems

1. Apply HashingTF instead of TF-IDF. Compare model performance and feature space size.

2. Increase training data from 500K to 5M records and plot the learning curve (accuracy vs dataset size).

2. Implement a pipeline using Spark ML Pipeline API (Tokenizer -> StopWordsRemover -> HashingTF -> IDF -> LogisticRegression).

Tools and Technologies Used:

Tool/Technology	Purpose
Apache Hadoop HDFS	Distributed storage of the SMS dataset

Apache Spark	Distributed data processing and computation
PySpark	Python interface for Apache Spark
Spark MLlib	Machine learning and text feature processing
Python	Implementation of the classification algorithm
TF-IDF	Conversion of text messages into numerical features
Logistic Regression	Classification of SMS messages as spam or legitimate
Ubuntu on WSL	Execution environment
Terminal and text editor	Writing, executing, and debugging the program

Conclusion

The SMS spam classification system was implemented using Apache Spark and PySpark. The dataset was loaded from HDFS and preprocessed using tokenization, stop-word removal, and TF-IDF feature extraction. Logistic Regression was trained using mini-batch gradient descent, and hyperparameters were tuned to improve model performance. The classifier was evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC. Experiments were also performed to analyze the relationship between dataset size, training time, and accuracy, and to compare full-batch and mini-batch gradient descent. This practical demonstrates how distributed computing can support scalable text classification and automated spam detection.